

Elbasan Pulse

One place for Elbasan's municipal data. Department exports become KPIs, trends and answers that leadership can trust.



Team	FirmatGroup
Event	AI4Society Youth Innovation Hackathon, Tirana, 25-27 September 2026
Track	D: Municipal Data Integration & Decision-Support Dashboard
Client	Municipality of Elbasan (Bashkia Elbasan)
Code version	Branch <code>codex/civic-interface</code> , commit 5a628d2, API v0.1.0
Document date	25 September 2026

Contents

01	Executive summary	Problem, users, solution, status
02	Technology stack	Languages, frameworks, libraries, tooling
03	System architecture	Current and target architecture, repository layout
04	Backend (API)	FastAPI service, configuration, LLM client, warehouse
05	Frontend (web)	Next.js app, routes, components, i18n, theming
06	Business logic	KPI definitions, filters, alerts, import rules, copilot, briefing
07	Data model	Sample dataset and target canonical model
08	Design system & UX	Identity, colour, type, motion, accessibility, screens
09	Security, privacy & AI governance	
10	Quality: testing, linting, CI	
11	Running & deploying	Local setup, verified run, Docker, hosting
12	Roadmap & known gaps	
13	Glossary	

How to read this document

Features are tagged by maturity: **Implemented** works in the code today, **Prototype** is a working UI driven by synthetic sample data, and **Planned** is designed in the spec but not yet built. Everything below was checked against the source code in the repository.

1. Executive summary

The problem

Municipal information in Elbasan is spread across separate department systems (public services, urban planning, social services, finance). Leadership, department heads and planning staff cannot see one unified picture of services, operations and citizen requests. That limits planning, performance monitoring and evidence-based decisions.

Who it is for

User	Primary need
Municipal leadership (Mayor, cabinet)	One-glance health of the municipality; a monthly briefing; ask questions without needing an analyst
Department heads	Their department's performance against targets; backlog and resolution trends
Planning & reporting staff	Onboard new department exports quickly; produce reports; export data through an API

The solution: integration-first decision support

Most dashboards assume the data is already unified. Elbasan Pulse treats **integration as the product**: any department export (CSV/XLSX) can be onboarded in minutes. AI proposes how its columns map to one shared data model and a person confirms it. On top of the unified data sit KPIs, trend and anomaly signals, a question-answering copilot that always shows the SQL and source behind each answer, a monthly leadership briefing in Albanian and English, and an open REST API.

Track D brief requirement	Elbasan Pulse feature	Status
Data fragmented across systems	Smart Ingest: AI-assisted mapping of exports into a canonical model	Prototype
Unified dashboard	KPI overview with month and department filters	Prototype
Performance monitoring	Department scorecards against a 90% on-time target	Prototype
Reporting and analytics	Monthly briefing (print or Markdown download)	Prototype
Trend analysis	Trend chart; Signals engine for anomalies	ChartSignals
Decision-support tools	Ask-the-data copilot with visible SQL and lineage	Prototype
APIs for interoperability	REST API at <code>/api/v1</code> , OpenAPI docs at <code>/docs</code>	Implemented (health)

Current status at a glance

Honest scope statement

The repository contains a pre-event scaffold plus a polished front-end prototype. All dashboard figures come from one explicitly labelled synthetic dataset. The CSV preview runs entirely in the browser and saves nothing. Sample answers are deterministic calculations, not AI output, and their SQL is illustrative. The API currently serves a health endpoint with a real DuckDB check and a production-ready Claude client wrapper, but ingestion, metrics endpoints, the copilot and signals are to be built during the hackathon on the organisers' data.

Three innovation claims the product is designed around:

1. Onboarding a new department's data is a guided, minutes-long task, not an IT project.
2. Every AI answer is auditable: the generated SQL and source tables are shown next to it.
3. It is transferable: any municipality can plug in its own exports; the canonical model and saved mappings are reusable.

2. Technology stack

Elbasan Pulse is a **monorepo** with two applications: a Python backend in `api/` and a TypeScript frontend in `web/`. They talk only over HTTP/JSON, so either side can be replaced or deployed independently.

2.1 Programming languages

Language	Version	Where and why
Python	3.12+	The whole backend (<code>api/</code>). Chosen for data work: fast to write, first-class DuckDB and Anthropic SDKs, strong typing via type hints and Pydantic.
TypeScript	5.x	The whole frontend (<code>web/src</code>). JavaScript with static types, so wrong props or data shapes are caught at build time.
SQL	DuckDB dialect	Warehouse queries and metric views; the copilot will generate read-only SQL.
CSS	Tailwind v4 + custom CSS	Styling; about 2,100 lines of hand-written design tokens and component styles in <code>globals.css</code> .
JSON	n/a	Translation catalogues (<code>messages/sq.json</code> , <code>messages/en.json</code>) and API payloads.

2.2 Backend frameworks and libraries (`api/`)

Technology	What it is	Role in Elbasan Pulse
FastAPI 0.115+	Modern Python web framework for building REST APIs. Uses type hints to validate requests and auto-generates OpenAPI documentation.	Defines the <code>/api/v1</code> routes, dependency injection of DB connections, CORS, and the Swagger UI at <code>/docs</code> .
Uvicorn 0.32+	High-performance ASGI server (runs async Python web apps).	Serves the FastAPI app on port 8000, with hot reload in development.
DuckDB 1.1+	An embedded analytical (OLAP) database, "SQLite for analytics". Runs in-process and stores data in a single file; very fast at aggregations and reads CSV natively.	The data warehouse (<code>data/pulse.duckdb</code>). No separate database server to run or administer.
Pydantic Settings 2.6+	Typed configuration loaded from environment variables and <code>.env</code> files.	The <code>Settings</code> class: API key, DB path, CORS origins, LLM budget.
Anthropic Python SDK 0.40+	Official client for the Claude API.	Wrapped by <code>LLMClient</code> for column mapping (Claude Haiku 4.5) and copilot/briefings (Claude Sonnet 5).
python-multipart	Parser for multipart form uploads.	Needed for upcoming file-upload (ingest) endpoints.

2.3 Frontend frameworks and libraries (`web/`)

Technology	What it is	Role in Elbasan Pulse
Next.js 16.3 (App Router, Turbopack)	The leading React framework: file-based routing, server components, server-side rendering, API route handlers and production bundling.	Pages <code>/</code> , <code>/ingest</code> , <code>/ask</code> , <code>/briefing</code> ; the <code>/briefing/download</code> route handler; standalone output for Docker.

React 19.2	UI library for building interfaces from components and state.	All screens. The React Compiler is enabled to auto-memoise components.
Tailwind CSS 4	Utility-first CSS framework (classes like <code>flex gap-3</code>).	Layout utilities, alongside custom civic CSS classes and design tokens.
shadcn/ui + Radix UI	shadcn/ui is a collection of copy-in components built on Radix, a library of unstyled, accessible primitives.	Sheet (side panels), dropdown menu, tooltip, sidebar, badge, button: keyboard and screen-reader accessible.
next-intl 4	Internationalisation for Next.js.	Albanian (default) and English, stored in a cookie; formatted, pluralised messages.
next-themes	Theme switching for Next.js without a flash of the wrong theme.	Light (default), dark and system themes.
lucide-react	Open-source line icon set.	All UI icons (departments, navigation, status).
class-variance-authority, tw-animate-css	Helpers for component style variants and CSS animations.	Button/badge variants and sheet transitions.
Apache ECharts (+ echarts-for-react)	Charting library.	Installed for future charts. <i>Not used yet:</i> the current trend chart is a hand-built, responsive SVG component.
Source Sans 3, Geist Mono	Google Fonts, self-hosted by <code>next/font</code> .	Civic body typeface and monospace for SQL.

2.4 Tooling, infrastructure and AI

Tool	Purpose
uv (Astral)	Fast Python package and virtual-environment manager; lockfile <code>uv.lock</code> guarantees reproducible installs.
pnpm 10.22	Fast, disk-efficient Node.js package manager; lockfile <code>pnpm-lock.yaml</code> .
Node.js 22	JavaScript runtime used to build and run the frontend and its tests.
pytest + httpx	Backend unit and HTTP tests (FastAPI <code>TestClient</code>).
node:test	Node's built-in test runner for frontend business logic (no extra framework).
Ruff	Python linter and formatter (rules E, F, I, UP, B, SIM; line length 100).
ESLint 9 + eslint-config-next	TypeScript/React linting.
Docker + Docker Compose	Container images for API and web; one command starts the whole system.
GitHub Actions	CI: lint, format check, tests and production build on every push and PR.
Make	Shortcut commands: <code>make install / dev / test / lint</code> .
Claude API (Anthropic)	In-product AI: <code>claude-haiku-4-5</code> for column mapping, <code>claude-sonnet-5</code> for SQL generation and briefings.
Claude Code, ChatGPT	Development assistants (disclosed in Annex E): scaffolding and implementation; UI design of the civic prototype.

3. System architecture

3.1 Target architecture

The approved design is a classic three-layer system: **ingest** turns messy exports into canonical tables, an **analytics layer** (metrics, copilot, signals) reads from the warehouse, and a **REST API** feeds the web dashboard and any external system.

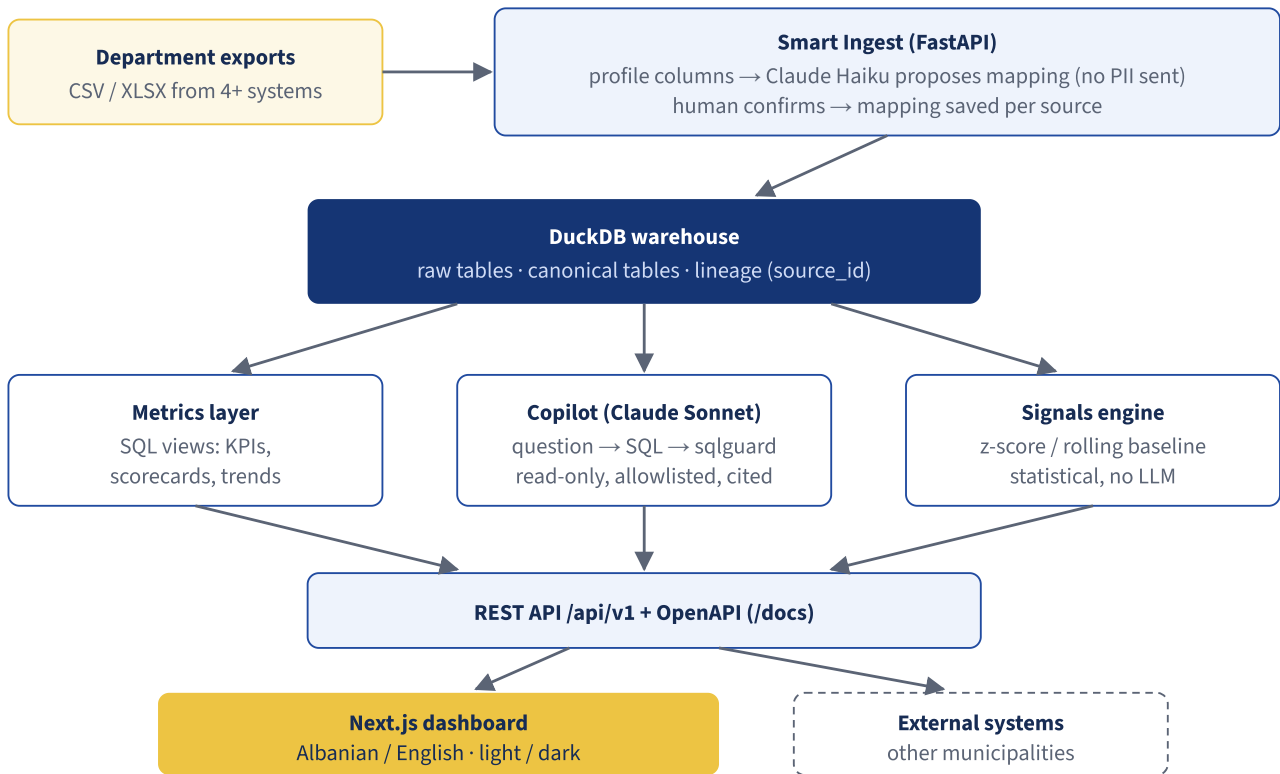


Figure 1. Target architecture from the approved design spec. The Briefing module (not drawn) combines metrics and signals and asks Claude Sonnet for a narrative.

3.2 What runs today

api/ Implemented

- `GET /api/v1/health`: runs `select 1` on DuckDB and reports DB and AI availability
- OpenAPI JSON and Swagger UI
- CORS for the web origin
- `LLMClient` with budget tracking and graceful failure (not yet called by any endpoint)

web/ Prototype

- Four screens: Overview, Sources, Ask the data, Monthly briefing
- All figures computed from `lib/pulse-data.ts` (synthetic)
- Live API status badge polling `/health` every 30 s
- Server route producing a Markdown report

Resilience principle: if the AI is unavailable or over budget, the dashboard, signals and API keep working. Only the copilot and the briefing narrative depend on the LLM; ingest falls back to manual column mapping.

3.3 Repository layout

```
elbasan-pulse/
├─ api/                                Python backend
│   ├── app/main.py                   FastAPI app factory, /api/v1 router, /health
│   ├── app/config.py                 Settings (env / .env), cached with lru_cache
│   ├── app/llm/client.py             LLMClient: model choice, spend tracking, safe errors
│   ├── app/warehouse/db.py          DuckDB connect + per-request dependency
│   ├── tests/                        pytest: health, db, llm (7 tests)
│   └─ pyproject.toml, uv.lock, Dockerfile
├─ web/                                Next.js frontend
│   └─ src/app/                       routes: page.tsx (/), ingest/, ask/, briefing/,
briefing/download/
│   └─ src/components/pulse/          overview, trend-chart, sources, csv-import, ask, briefing,
details-sheet
│   ├── src/components/ui/            shadcn/Radix primitives
│   ├── src/lib/                     pulse-data.ts (sample data + KPI maths), csv-preview.ts, api.ts
│   ├── src/i18n/                    locale config, cookie server action, request config
│   ├── messages/                    sq.json, en.json
│   └─ tests/pulse.test.mjs          node:test (9 tests)
├─ docs/                              design specs, plans, pitch, Annex E declaration
├─ .github/workflows/ci.yml           CI pipeline
└─ docker-compose.yml, Makefile, .env.example
```

4. Backend (API)

4.1 Application structure

`app/main.py` uses an **app factory** (`create_app()`) so tests can build a fresh app with different settings. Routes live on an `APIRouter` with prefix `/api/v1`, which gives the API a versioned, stable contract for external systems.

4.2 Endpoints

Endpoint	Behaviour	Status
GET <code>/api/v1/health</code>	Returns <code>{"status": "db", "llm", "version"}</code> . <code>db</code> is true when DuckDB answers <code>select 1</code> ; <code>llm</code> is true when an Anthropic API key is configured.	Implemented
GET <code>/docs</code> , <code>/openapi.json</code>	Interactive Swagger UI and the machine-readable OpenAPI schema.	Implemented
<code>/api/v1/ingest/*</code>	Upload, profile, propose mapping, confirm and load.	Planned
<code>/api/v1/metrics/*</code>	KPIs, scorecards and trends from metric views.	Planned
<code>/api/v1/ask</code> , <code>/signals</code> , <code>/briefing</code>	Copilot, anomaly alerts, generated briefing.	Planned

Verified response during this run:

```
GET http://localhost:8000/api/v1/health
{ "status": "ok", "db": true, "llm": false, "version": "0.1.0" }
```

`llm` is false because no `ANTHROPIC_API_KEY` was configured locally; this is the expected, graceful state.

4.3 Configuration

Variable	Default	Meaning
<code>ANTHROPIC_API_KEY</code>	<code>none</code>	Enables AI features; without it they report "AI off".
<code>DUCKDB_PATH</code>	<code>data/pulse.duckdb</code>	Warehouse file; parent folder is created automatically. <code>:memory:</code> is supported.
<code>LLM_BUDGET_USD</code>	<code>20</code>	Hard spending cap per process for Claude calls.
<code>CORS_ORIGINS</code>	<code>["http://localhost:3000"]</code>	Browser origins allowed to call the API.
<code>NEXT_PUBLIC_API_URL</code>	<code>http://localhost:8000</code>	(web) Base URL the frontend calls.

4.4 Warehouse access

`get_connection()` is a FastAPI *dependency* that opens one DuckDB connection per request and always closes it in a `finally` block. DuckDB is embedded, so there is no database server, credentials or network hop.

4.5 LLM client: a safety contract

`app/llm/client.py` is the single entry point for every Claude call. Its contract is that **the LLM layer never crashes the app**: callers always receive an `LLMResult(ok, text, error, input_tokens, output_tokens)`.

Situation	Result
No API key	<code>error = "llm_unavailable"</code> , no network call
Budget spent (<code>spent_usd ≥ LLM_BUDGET_USD</code>)	<code>error = "llm_budget_exhausted"</code> , no network call
Rate limited / HTTP error / connection error	<code>llm_error:rate_limited</code> , <code>llm_error:<status></code> , <code>llm_error:connection</code>
Model refusal	<code>error = "llm_refusal"</code> (tokens still counted)
Anything unexpected	Logged, returns <code>llm_error:unexpected</code>
Success	Concatenated text blocks plus token usage

Cost tracking: after every call, `spent_usd += (input_tokens × in_price + output_tokens × out_price) / 1,000,000` , using a price table per model (Haiku: \$1 / \$5 per million tokens; Sonnet: \$2 / \$10). A process-wide singleton (`get_llm()`) keeps spend across requests. Two model tiers are defined: `MODEL_FAST = claude-haiku-4-5` for cheap structured tasks such as column mapping, and `MODEL_SMART = claude-sonnet-5` for SQL and narrative.

5. Frontend (web)

5.1 Routes

Route	Screen	Contents
/	Overview	Three KPI tiles, trend chart, "Needs attention" alerts, department performance table, department detail sheets.
/ingest	Sources	Four source cards with freshness, source-to-result flow diagram, 3-step CSV import preview.
/ask	Ask the data	Question box, three sample questions, answer card with period, lineage sheet and SQL disclosure.
/briefing	Monthly briefing	Leadership report built from the same metrics; print and download actions.
/briefing/download	Route handler	Server-side GET returning a Markdown file (?locale=sq en) with Content-Disposition: attachment and nosniff .

5.2 Component architecture

- **Layout** (`app/layout.tsx` , server component): loads fonts, reads the locale, and wraps the app in `ThemeProvider` → `NextIntlClientProvider` → `TooltipProvider` → `AppShell` .
- **AppShell / AppSidebar**: persistent deep-blue sidebar with the four destinations, the active item marked with `aria-current="page"` ; on phones it collapses into a menu. The top bar holds breadcrumbs, a "Sample data" badge, the language switch and the theme toggle.
- **Pulse feature components** (`components/pulse/`): `Overview` , `TrendChart` , `Sources` , `CsvImport` , `Ask` , `Briefing` , `DetailsSheet` . They are client components (`"use client"`) holding local React state for filters, steps and answers.
- **Pure logic modules** (`lib/`): `pulse-data.ts` and `csv-preview.ts` contain no React, which makes the business rules unit-testable with plain Node.
- **ApiStatus**: polls `/api/v1/health` every 30 seconds and shows "API online / AI on-off" or "offline".

5.3 Internationalisation

Albanian (`sq`) is the default; English (`en`) has equal coverage. The chosen locale is stored in the `NEXT_LOCALE` cookie for one year by a **server action** (`setLocale`), which validates the value against the allowed list. Numbers are formatted deterministically by a custom `numberFormatter` (Albanian: `1.248` and `87,4` ; English: `1,248` and `87.4`) instead of the browser's ICU data, which avoids server/browser hydration mismatches. Month abbreviations are localised too (`sht` / `Sep`).

5.4 Theming

`next-themes` applies a `class` on `<html>` . Light is the initial theme; dark and system are available. All colours are CSS custom properties (design tokens) redefined for dark mode, so components never hard-code colours.

5.5 Charting and build

The trend chart is a custom SVG component: a `ResizeObserver` measures its container and recomputes the path for any width (minimum 220 px). It supports point selection, a "view chart data" table alternative for accessibility, and a link to the data sources.

Build configuration: `next.config.ts` enables the **React Compiler** (automatic memoisation), **standalone output** (a minimal self-contained server for Docker), and the next-intl plugin.

6. Business logic

All rules below live in `web/src/lib/pulse-data.ts`, `csv-preview.ts` and the feature components. They are the reference behaviour the real metrics API must reproduce on the organisers' data.

6.1 KPI definitions

KPI	Definition
Received	Number of citizen requests created in the period.
Resolved	Requests closed in the period.
On time	Resolved requests closed within their deadline.
Overdue	Open requests past their deadline.
On-time rate	$\text{on_time} / \text{resolved} \times 100$ (0 when nothing was resolved). For "all departments" it is a <i>weighted</i> rate: department counts are summed first, then divided. Averaging department percentages would be wrong.
Change vs previous	Received: $(\text{current} / \text{previous} - 1) \times 100$ %. On-time: difference in percentage points.

Fair comparison rule

The period is 1-24 September 2026, and it is compared with the same 24 days of August ("the same 24 days of each month are compared"), so a partial month is never compared with a full one.

6.2 Filters and aggregation

`getMetrics(department, period)` sums the four counters over the selected department, or over all four when the filter is "all", and derives the rate. The overview recomputes every tile, alert and table row whenever the **month** (September / August) or **department** filter changes.

`getTrend()` returns monthly received/resolved values from April. April-July are city totals split across departments by fixed shares (public 39%, urban 25%, social 21%). **Finance receives the rounding remainder** so the four departments always add up exactly to the city total; a unit test enforces this.

6.3 Target and alert rules

Rule	Logic
Performance target	A department is On target when its on-time rate is $\geq 90\%$, otherwise Below target .
Stale source	Each source carries a freshness flag. In the sample, Urban planning (last updated 18 Sep, six days before the period end) is flagged Needs update , and in September that department's status shows "stale" regardless of its rate. The flag is set in the sample data today; the planned version derives it from the source's upload date.
Overdue queue alert	Shown when the filter includes Public services: "28 requests past due", because that department holds the largest queue.
Stale-source alert	Shown in September when the filter includes Urban planning.
City-level alert	If the overall rate is $\geq 90\%$: "No alerts", otherwise "Below target".

6.4 CSV import preview (Smart Ingest prototype)

Three steps: **Choose file** → **Check columns** → **Review**. The file is read with `File.text()` in the browser; nothing is uploaded or saved.

Validation	Rule
File type	Must end in <code>.csv</code>
Size limits	$\leq 1,000,000$ bytes, ≤ 50 columns, $\leq 10,000$ rows
Parsing	RFC-4180 style: quoted fields, escaped <code>"</code> quotes, commas and newlines inside quotes, UTF-8 BOM removal, CRLF or LF. Text after a closing quote or a stray quote is rejected as ambiguous.
Structure	A header plus at least one data row; headers non-empty and unique (case-insensitive); every row has exactly as many cells as the header.
Auto-mapping	Column names are matched in Albanian or English: <code>request_id id</code> → Request ID, <code>department drejtoria</code> → Department, <code>status statusi</code> → Request status.
Mapping validity	All three fields must be mapped, to valid and <i>distinct</i> columns.
Race safety	A generation counter ignores results from a file read that was superseded by a newer choice.

Error messages are specific and localised: wrong type, too large, empty, bad headers, malformed.

6.5 Ask the data (copilot prototype)

- Three supported sample questions (most overdue department, on-time performance, request volume). Input is normalised (trim, lower-case, trailing `?!.` removed) and compared with the translated question in the current language.
- Answers are **calculated** from `getMetrics`, for example "Public services has 28 overdue requests. That is 66.7% of the 42 overdue requests."
- Each answer carries the period, a "View sources" lineage sheet and a "View query" disclosure with the SQL it corresponds to (illustrative).
- **Unknown questions never get a made-up answer.** They receive an explanatory "not available" message. Empty input is rejected with an inline error.

In the planned version, Claude Sonnet receives only the schema of allowlisted views, writes SQL, and `sqlguard` accepts only one read-only `SELECT` with `LIMIT  $\leq 1000$` and a 5-second timeout before execution.

6.6 Monthly briefing

Built from the same metrics: executive summary (received, resolved, on-time rate, overdue), a priority section (the public-services queue), a data-quality section (the stale source) and the list of connected sources. It can be printed, or downloaded as Markdown in Albanian or English from `/briefing/download`. Every version is labelled a draft based on sample data.

7. Data model

7.1 Synthetic sample dataset Prototype

Aggregate-only fixtures. They contain no records about people and must never be presented as municipal data.

Department	Period	Received	Resolved	On time	Overdue	On-time rate
Public services	1-24 Sep	486	420	328	28	78.1%
Urban planning	1-24 Sep	312	280	246	9	87.9%
Social services	1-24 Sep	264	256	249	3	97.3%
Finance	1-24 Sep	186	180	170	2	94.4%
All departments	1-24 Sep	1,248	1,136	993	42	87.4%
All departments	1-24 Aug	1,182	1,048	883	34	84.3%

Derived headline values: received +5.6% vs August; on-time rate +3.2 percentage points.

Four sample sources: sherbimet_publike_shtator.csv (486 rows), urbanistika_shtator.csv (312, stale since 18 Sep), sherbimet_sociale_shtator.csv (264), financa_shtator.csv (186).

7.2 Target canonical model Planned

To be finalised against the organisers' data package; the architecture does not change if fields do.

Table	Columns
department	id, name_sq, name_en
service_category	id, department_id, name_sq, name_en
request	id, source_id, category_id, department_id, location_id, created_at, closed_at, status, channel
location	id, name, lat, lon, zone
budget_line	id, department_id, period, planned, actual
kpi_target	id, department_id, metric, period, target
source	id, name, filename, uploaded_at, mapping_json: lineage for every row



Figure 2. Canonical model relationships. Arrows point from a table to the table it references.

7.3 Planned data flows

1. **Ingest:** upload → profile (types, samples, null %) → Haiku receives column names, the profile and a few non-personal sample values → returns mapping JSON → user edits and confirms → rows are written to canonical tables with `source_id`.
2. **Dashboard:** web calls `/api/v1/metrics/*` → metric views.
3. **Copilot:** question → Sonnet sees allowlisted view schemas only → SQL → sqlguard → execute → aggregates → Sonnet writes a short answer plus chart spec → UI shows answer, chart, SQL and sources.
4. **Briefing:** metrics + signals → Sonnet → Markdown briefing (sq/en).

8. Design system & UX

8.1 Identity

A calm, trustworthy civic workspace in Elbasan's familiar *verdhëblu* (yellow-blue) colours, adapted for screens. These are inspired shades, not official municipal colour specifications. The brand mark combines a civic tower with a short pulse line.

Token	Light	Dark	Use
Sidebar / identity blue	 #153574	 #1d232b panels	Navigation
Golden yellow	 #edc443 with #172c54 text	 #d7b768 muted gold	Selected navigation, highlights
Primary action	 #214ba0	 #345c98	Filled buttons (white text)
Canvas	 #f7f7f3 stone	 #15191f charcoal	Page background
Chart series	 #2454ad /  #a07812	soft blue / gold	Received / resolved (darker gold for contrast)
Status	Green = on target · amber = warning (semantic colours kept)		Status labels

Typography: Source Sans 3 (Latin + Latin Extended for ë and ç), navy text. **Iconography:** plain line icons without decorative tiles or circles. **Copy:** simple punctuation, short hyphens in date ranges, localised labels for empty CSV values.

8.2 Motion and accessibility

- Motion only responds to meaning: selection, disclosure and import progress, plus a brief trace on the brand mark. There are no endless decorative animations.
- `prefers-reduced-motion` users receive static equivalents.
- Full keyboard navigation; side sheets trap focus and close with Escape; the import step heading receives focus after each step.
- Live regions (`aria-live`) announce KPI changes and answers; errors use `role="alert"` and `aria-invalid` .
- The chart has a data-table alternative; icons are `aria-hidden` ; filters have screen-reader labels.
- Responsive down to phone widths; "Sample data" is always visible so synthetic figures cannot be mistaken for real ones.

8.3 Screens

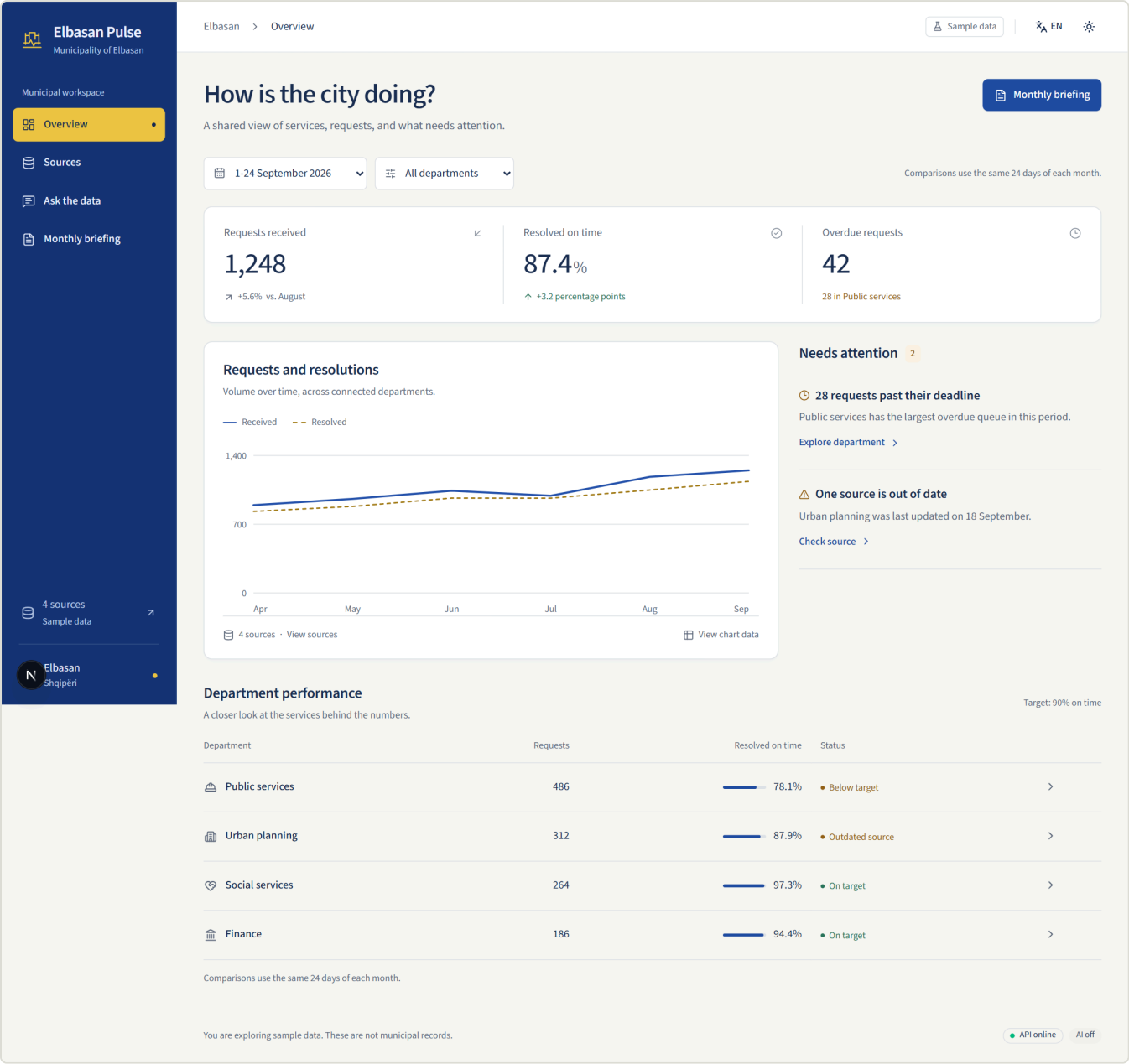


Figure 3. Overview (English, light): KPI tiles, trend chart, "Needs attention" alerts and department performance.

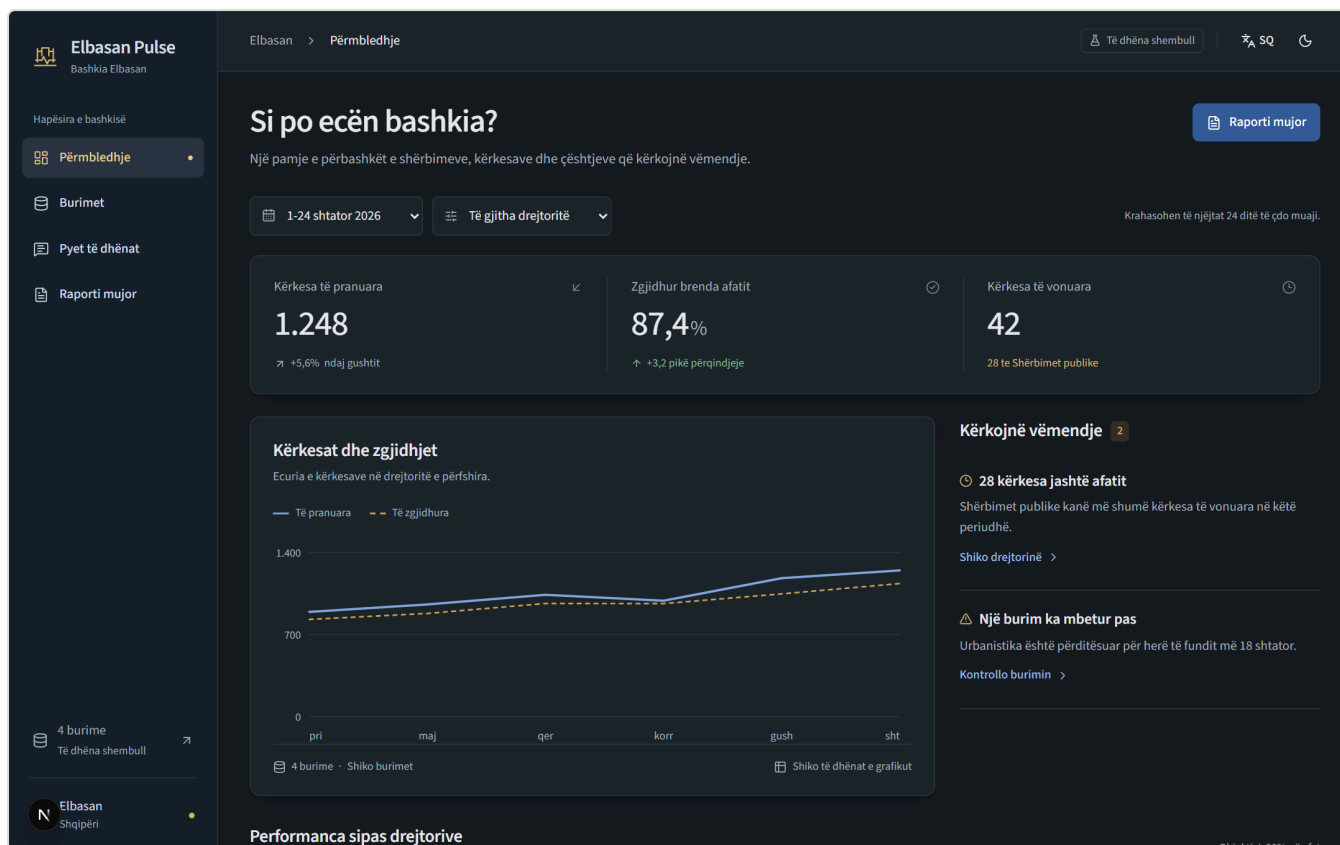


Figure 4. Overview in Albanian with the dark theme.

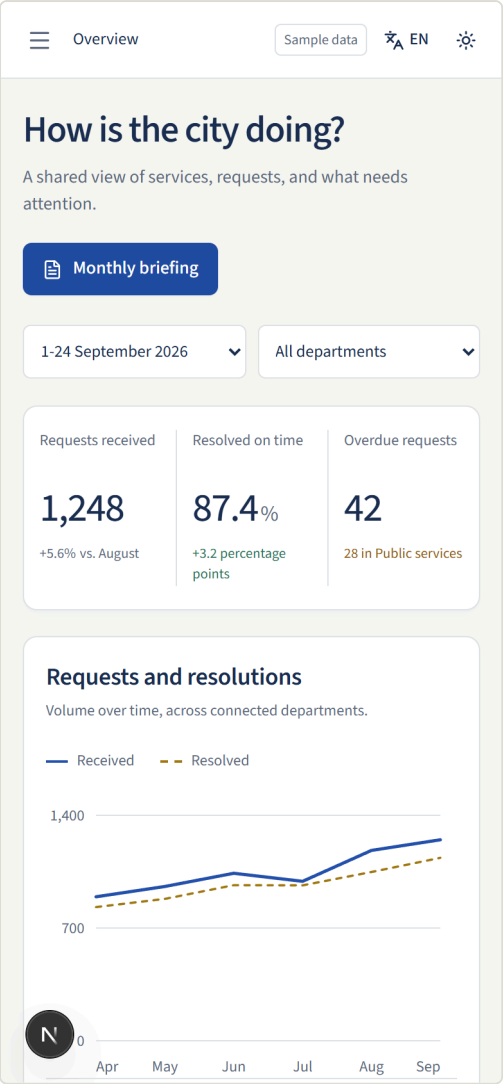


Figure 5. Phone width (390 px).

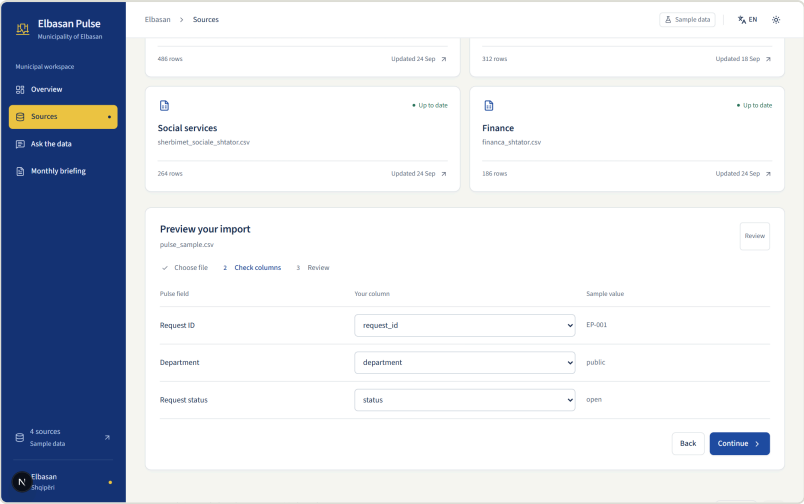


Figure 6. CSV import, step 2: columns automatically mapped to Pulse fields, with sample values.


Elbasan Pulse
Municipality of Elbasan

Municipal workspace

Overview
Sources
Ask the data
Monthly briefing

4 sources
Sample data


Elbasan
Shqipëri

Elbasan > Ask the data

Sample data
EN

A question. An answer you can trace.

Explore the sample data, then see the calculation behind the answer.

What would you like to understand?

Which department has the most overdue requests?

Explore answer

Start with a sample question

Which department has the most overdue requests?
How many requests were resolved on time?
How many requests did we receive this month?

Calculated sample answer · not generated by AI

Which department has the most overdue requests?

Public services has 28 overdue requests.

That is 66.7% of the 42 overdue requests across all departments. This is the largest queue in the sample dataset.

1-24 September 2026
View sources

View query

Evidence, always included

The source, period, and calculation stay beside the answer.

Calculated sample answer · not generated by AI

You are exploring sample data. These are not municipal records.

API online
AI off

Figure 8. Ask the data: a calculated sample answer with period, sources and query. The footer shows the live API status ("API online · AI off").

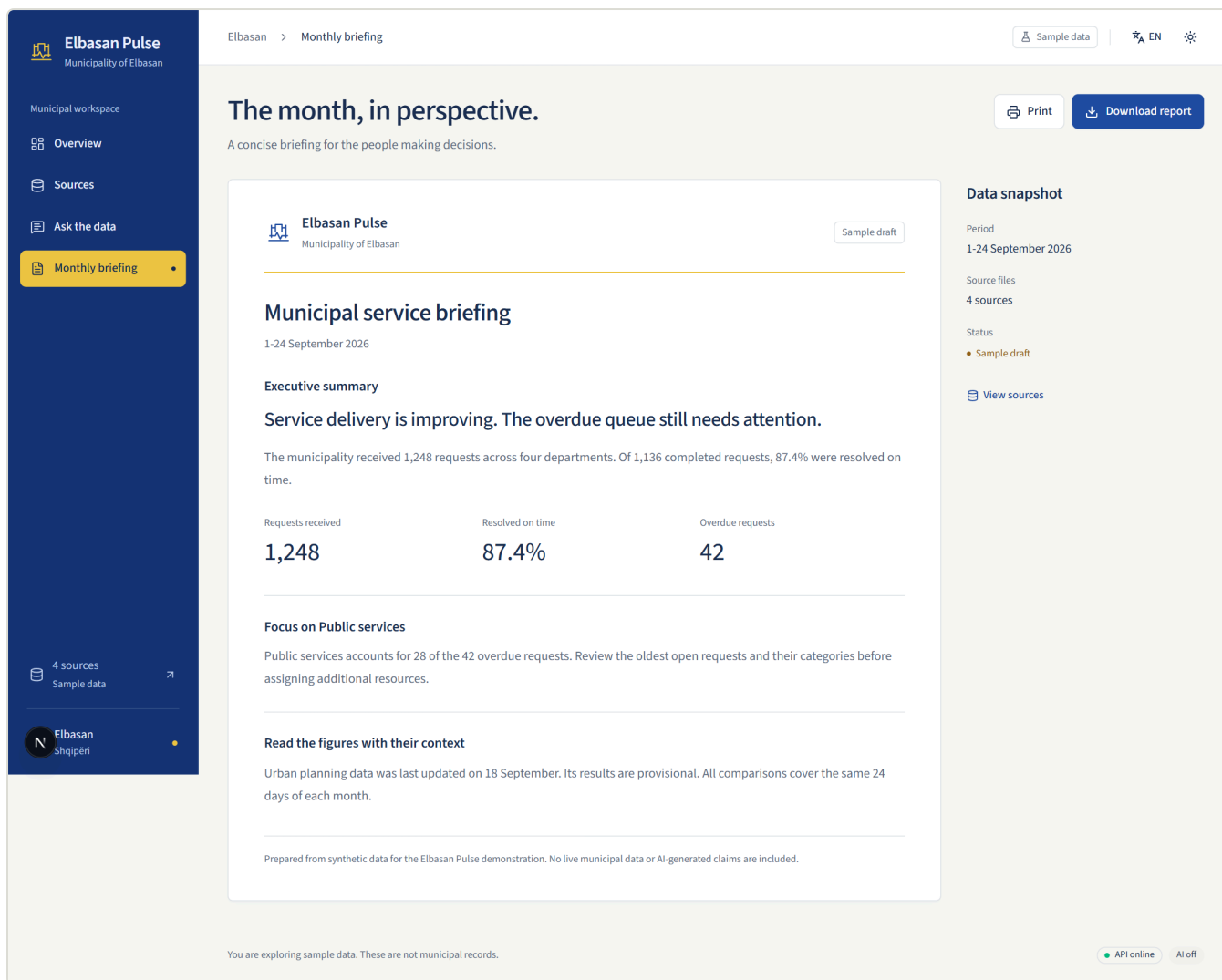


Figure 9. Monthly briefing with print and Markdown download.

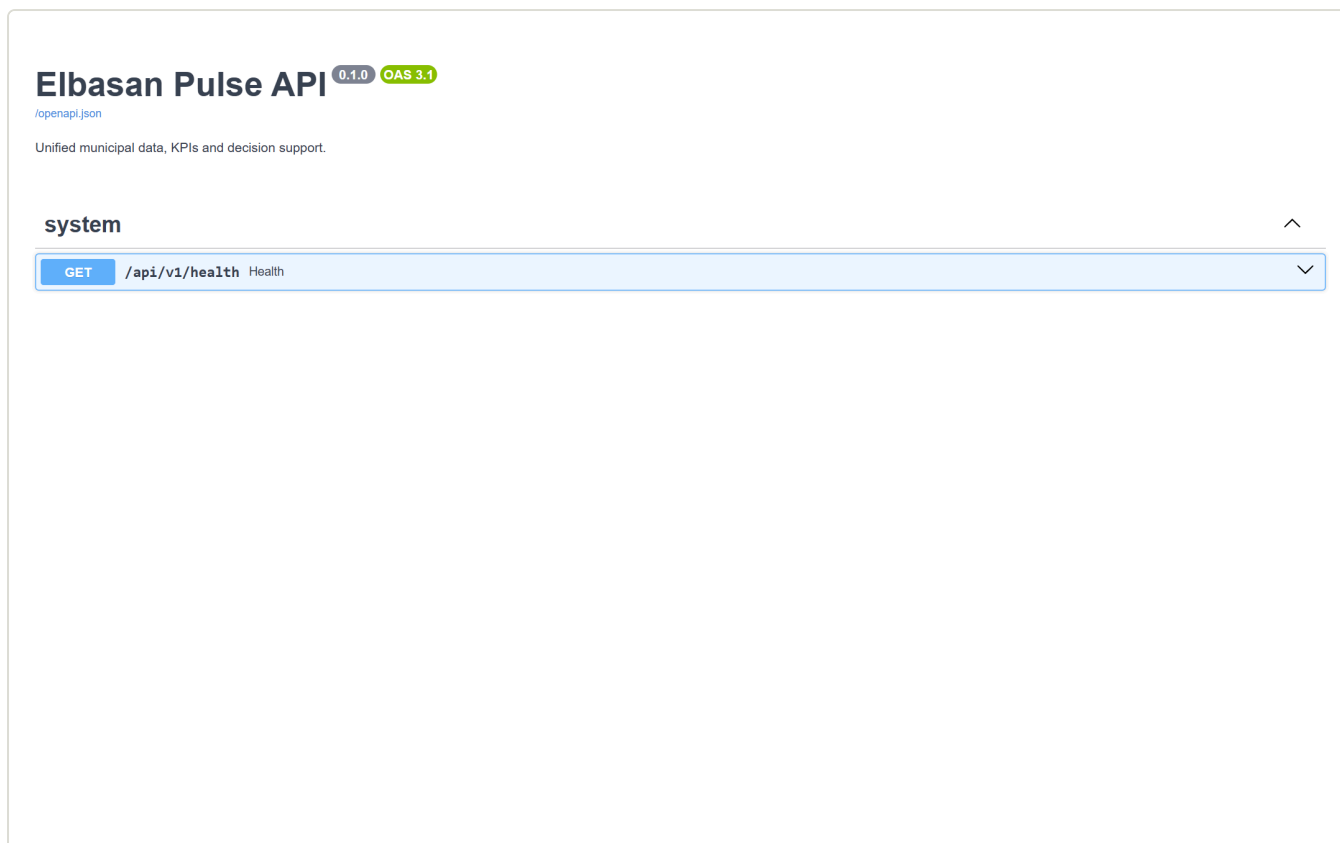


Figure 10. Auto-generated OpenAPI (Swagger) documentation at <http://localhost:8000/docs>.

9. Security, privacy & AI governance

Area	Measure
Data sources	Only organiser-provided or approved data is used; no real personal data. The prototype uses only synthetic aggregates.
PII and the LLM	Personal fields (names, phones, emails, ID numbers) are flagged by the profiler and never sent to Claude; only column names, profiles and non-personal samples are.
Browser CSV preview	Files are parsed locally; nothing is transmitted or stored; state is session-local.
Copilot SQL guard	Single read-only <code>SELECT</code> over allowlisted views only, <code>LIMIT ≤ 1000</code> , 5 s timeout; rejected SQL returns a clear message rather than a guess.
Hallucination control	Answers always show SQL and sources; unsupported questions get "not available" instead of a fabricated answer.
Cost control	Hard USD budget in <code>LLMClient</code> ; calls stop once it is reached.
Availability	Every LLM failure is caught and mapped to an error code; the non-AI features keep working.
Web hardening	CORS restricted to configured origins; locale cookie validated server-side with <code>SameSite=Lax</code> ; download served with <code>X-Content-Type-Options: nosniff</code> .
Secrets	API key only in environment / <code>.env</code> (git-ignored); <code>.env.example</code> documents variables without values.
Transparency	Annex E declares all AI tools (Claude Code, ChatGPT, Claude API models), open-source libraries and the pre-event components, with git history as proof.

Out of scope: user accounts / SSO, write-back to source systems, real-time connectors to live municipal systems (named as next steps for the mentoring phase), and a mobile app.

10. Quality: testing, linting, CI

10.1 Test results from this run

Suite	Command	Result
API (pytest)	<code>uv run pytest -q</code>	7 passed
Web (node:test)	<code>node --experimental-strip-types --test tests/*.test.mjs</code>	9 passed

10.2 What is covered

API

- Health returns ok, `db: true`, `llm: false` without a key
- OpenAPI schema is served
- DuckDB connects and runs a query
- LLM: unavailable without key, spend tracking, SDK errors caught, budget blocks calls (fake SDK, no network)

Web business logic

- City totals reconcile with departments; weighted on-time rate
- Filters change the dataset consistently
- Historical trend reconciles in every month
- CSV: quotes, escaped quotes, BOM, CRLF, multiline cells, malformed and ambiguous input, empty fields, byte/row/column limits
- Mapping requires unique columns for all fields
- Albanian formatting is identical on server and browser

10.3 Continuous integration

GitHub Actions (`.github/workflows/ci.yml`) runs on every push and pull request, in two parallel jobs:

- **api:** `uv sync --frozen` → `ruff check` → `ruff format --check` → `pytest`
- **web:** `pnpm install --frozen-lockfile` → `pnpm lint` → `pnpm build` (type-checks the whole app)

Planned additions from the spec: sqlguard allow/deny tests, mapping and metrics tests on a fixture dataset, signals on synthetic series, a ~20-question copilot golden set with honestly reported accuracy, and a Playwright smoke test of ingest → dashboard → ask.

11. Running & deploying

11.1 Prerequisites

Python 3.12+ with **uv**; Node.js 22 with **pnpm** 10.22; optionally Docker Desktop and an Anthropic API key.

11.2 Local development

```
cp .env.example .env          # add ANTHROPIC_API_KEY (optional)
cp .env.example api/.env
make install                   # uv sync + pnpm install
make dev                       # API on :8000, web on :3000 (in parallel)

# or separately
cd api && uv run uvicorn app.main:app --reload --port 8000
cd web && pnpm dev
```

Open **http://localhost:3000** for the dashboard and **http://localhost:8000/docs** for the API. The UI works without the API; the status badge then shows "offline".

11.3 Verified run (25 Sept 2026, Windows 11)

Step	Outcome
API install (<code>uv sync</code>)	OK
API server	Running on :8000; <code>/health</code> returned <code>db: true, llm: false</code>
Web install	OK after installing pnpm 10.22.0 globally (it was not on the PATH)
Web server	Next.js 16.3.6 (Turbopack) ready on :3000; all four screens render in both languages and themes; footer shows "API online · AI off"
Tests	API 7/7, web 9/9

Setup notes found during the run

- `pnpm test` fails on Node 22.14 with `ERR_UNKNOWN_FILE_EXTENSION ".ts"` because the tests import TypeScript files directly. Node ≥ 22.18 strips types by default; on older versions run with `--experimental-strip-types` (or add the flag to the `test` script). CI does not run `pnpm test` today, so this is not caught automatically.
- pnpm must be installed (for example `npm i -g pnpm@10.22.0` or `corepack enable`).

11.4 Docker

```
docker compose up --build      # api :8000, web :3000
```

- API image:** `ghcr.io/astral-sh/uv:python3.12-bookworm-slim`, frozen production dependencies, Uvicorn on 0.0.0.0:8000. The warehouse is kept on a named volume `pulse-data` at `/data/pulse.duckdb`.
- Web image:** two-stage `node:22-alpine` build; the runtime stage contains only the Next.js standalone server and static assets. `NEXT_PUBLIC_API_URL` is baked in at build time.

Hosting plan: web on Vercel; API on Render or Hugging Face Spaces (Docker). A localhost demo via Docker Compose is the accepted fallback.

12. Roadmap & known gaps

12.1 Build plan

Tier	When	Contents
Scaffold	Before 13:00 Fri (done)	Monorepo, Docker, CI, FastAPI health + DuckDB, LLM wrapper, Next.js shell; civic UI prototype on synthetic data. Declared in Annex E.
Must-have (CP2)	Sat 15:00	Smart ingest, canonical model, KPI dashboard on real data, copilot with visible SQL
Stretch (CP3)	Sat 21:00	Signals, AI briefing, map, OpenAPI polish, role views
Freeze	Sat 21:00 → Sun 11:00	Polish, tests, deck (≤ 8 slides), Annex E, tagged release

12.2 Known gaps and recommendations

1. **Replace synthetic data.** Implement `/api/v1/metrics/*` and switch the overview, ask and briefing screens from `pulse-data.ts` to API calls, keeping the same KPI definitions and tests.
2. **Server-side ingest.** Move CSV parsing and validation rules to the API (plus XLSX), add the PII profiler and Haiku mapping proposals.
3. **sqlguard module** (the spec names `sqlglot`) before exposing the copilot.
4. **Signals engine:** z-score / rolling-baseline alerts to replace the hard-coded alert rules.
5. **Web tests in CI:** add `pnpm test` to the workflow and the type-stripping flag to the script.
6. **Unused dependencies:** ECharts, `cn` and the `shadcn` CLI are listed as runtime dependencies; either use ECharts for new charts or remove them to slim the bundle.
7. **Price table:** verify the per-model prices in `PRICING` against current Anthropic pricing so the budget cap is accurate.

13. Glossary

Term	Meaning
API / REST	A way for programs to talk over HTTP using URLs and JSON. Elbasan Pulse exposes its data at <code>/api/v1</code> .
OpenAPI / Swagger	A standard, machine-readable description of an API, plus an interactive web page for trying it.
ASGI	The standard interface between async Python web apps and servers such as Uvicorn.
Canonical model	The one shared set of tables every department's data is mapped into, so it can be compared and combined.
Column mapping	Deciding which column in an export corresponds to which canonical field (for example <code>drejtoria</code> → department).
Lineage	Tracking which source file every number came from.
OLAP / warehouse	A database optimised for analytical queries (sums, averages, trends) rather than many small updates.
KPI	Key performance indicator, such as the on-time resolution rate.
LLM	Large language model; here Claude, used for mapping suggestions, SQL generation and briefings.
Server / client component	In Next.js, server components render on the server; client components (<code>"use client"</code>) run in the browser and hold interactive state.
Hydration	When React in the browser attaches to server-rendered HTML; server and browser output must match exactly.
i18n	Internationalisation: supporting multiple languages (here Albanian and English).
CI	Continuous integration: automatic lint, test and build on every code change.
PII	Personally identifiable information: names, phone numbers, emails, ID numbers.

Elbasan Pulse · FirmatGroup · AI4Society Youth Innovation Hackathon 2026. Generated from repository commit 5a628d2.